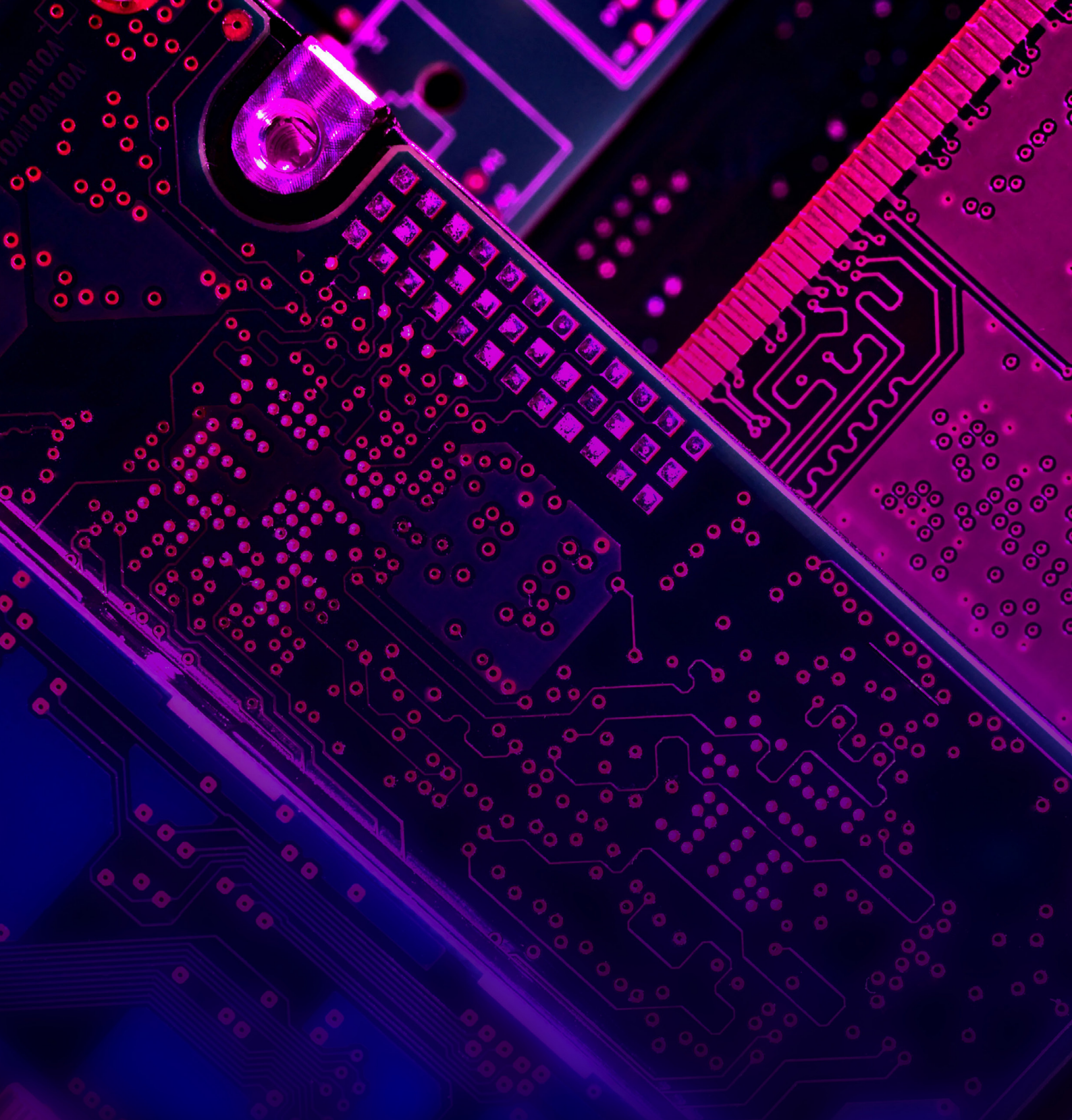


Research Priorities for AI Compute Verification

James Petrie

August 31, 2026

Table of Contents

Part I: What Could Be Verified 2

Why Verify AI Compute 3

What Would Need to Be Verified 4

Three Illustrative Verification Cases 5

Part II: What It Would Take 8

Selected R&D Recommendations 8

Quantifying Relevant Compute 9

Methods for collecting evidence on compute usage 11

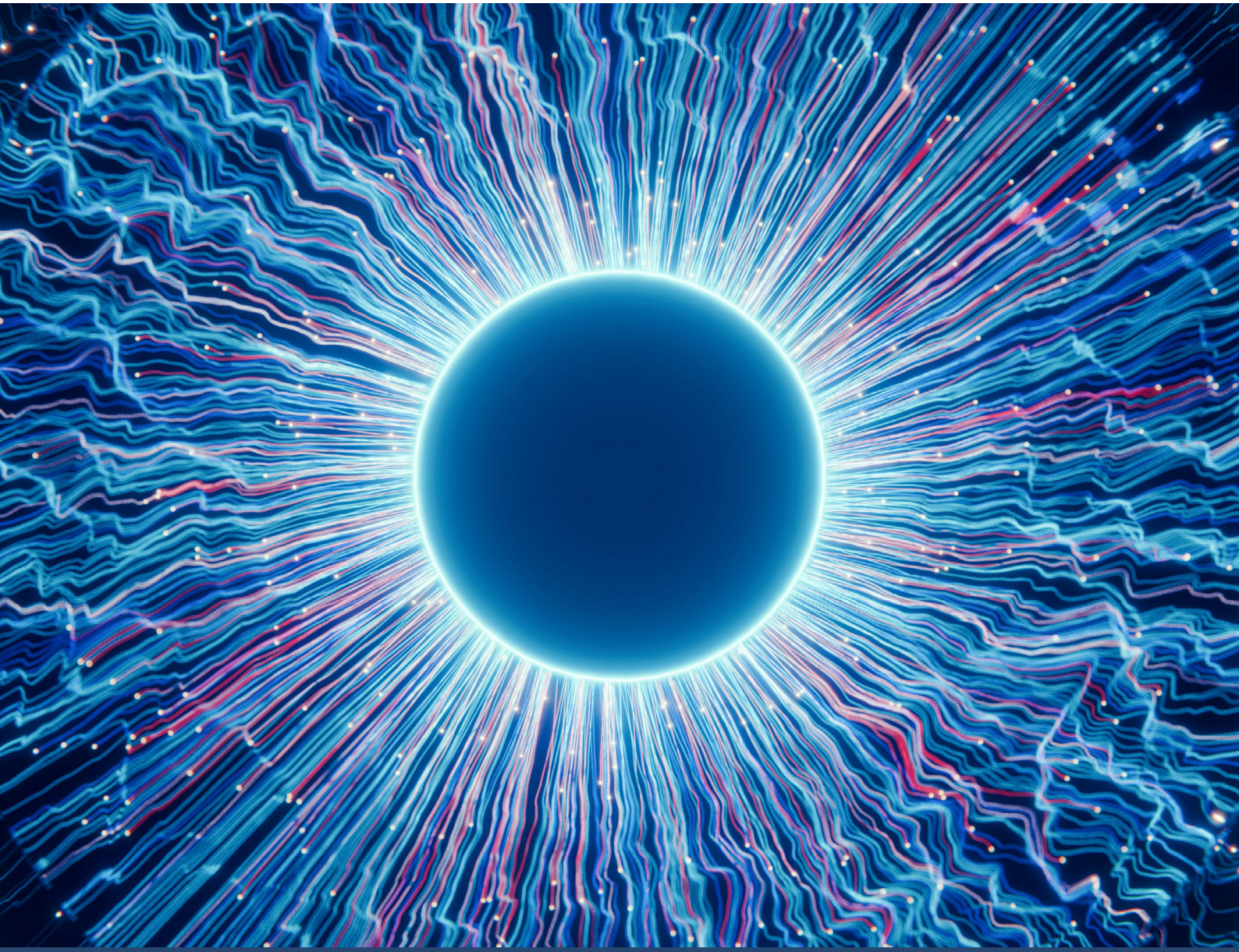
Infrastructure for Analyzing Evidence 18

Defending Against Physical Circumvention 22

Verification Protocols 24

Complementary Work Beyond Technical R&D 28

Conclusion 30



Part I: What Could Be Verified

This is an input paper prepared by James Petrie (Compute Security and Governance Lead at the Future of Life Institute). It has benefited from input and comments from Amro Awad, Mark Brakel, Naci Cankaya, Peter Drotos, Connor Dunlop, Jonathan Happel, Will Hodgkins, Halfdan Holm, Daan Juijn, Thomas Milton, Yannick Muhlhauser, Zac Richardson, Wayne Tee, Robert Trager, and José J. Villalobos, whose contributions are gratefully acknowledged. The document has been prepared as input for consultations amongst international policy makers and experts in the Netherlands.

The paper concerns high-stakes verification between two or more states of advanced AI development and usage. This type of verification must hold between parties who do not trust one another and who have strong incentives to circumvent it, and is a more demanding standard

than commercial audits or voluntary corporate transparency. Much of the engineering difficulty originates from this high standard.

The paper centers around compute as the object of verification. However, a stack of verification technology may involve hardware focussed technological options in combination with other methods such as: developers' own declarations checked by audit, interviews with named personnel, inspection of data centres and their power draw, monitoring of deployed systems through their interfaces, and national technical means and open-source intelligence. These complementary layers reduce the chance of an undetected breach and serve as placeholders while more advanced hardware-based tools are developed.

The approaches described below are at very different stages of maturity, and this bears directly on funding decisions. Some could be deployed within months if there were demand for them; others rest on research prototypes or on open mathematical questions. The paper provides reasoning on which is which, so that ready work is commissioned and early work is researched.

A brief glossary of terms used in compute verification

Compute: the advanced chips used for AI development.

Prover: the actor making claims about how compute is being used.

Verifier: the actor assessing whether the prover's claims are true or not.

Workload: The set of computations running on the compute, such as inference or training.

FLOP: short for floating point operation (the addition or multiplication of two numbers), the basic unit AI computation is measured in.

Attestation: A statement about the state or activity of a piece of hardware or software, generated in a way intended to make it difficult to falsify.

Inference: The use of an already-trained model to produce outputs: answering a query, generating text or code, operating as an agent. Distinguished from **training**, which is the process of creating or modifying the model in the first place.

Claim: A specific, checkable statement about compute or its use, which a verification protocol is designed to confirm or disconfirm.

Zero Knowledge Proof (ZKP): A cryptographic method by which one party can prove a statement is true without revealing anything beyond its truth.

Why Verify AI Compute

Advanced AI poses large-scale risks that threaten the national security of every state. These risks cross borders, so no state can manage them through domestic measures alone. States have historically embedded the rules for cooperation on such risks in international agreements.

Where the stakes are high, the scope of those agreements is determined by what can be verified. For example, negotiators sought a comprehensive nuclear test ban but could not agree on how to detect underground tests. The Limited Test Ban Treaty excluded those,

prohibiting only what could be verified through national technical means at the time: atmospheric, underwater and outer-space tests. By contrast, strengthening the Biological Weapons Convention has been handicapped by verification challenges. The presence or absence of a credible verification mechanism has been among the strongest determinants of whether a regime achieved its objective.

Compute is a promising option for AI verification because it's indispensable for AI development, very difficult to manufacture, and could potentially make automatic claims about the computations which developers run through them. This could potentially be done by, for example, using privacy-preserving proofs that demonstrate compliance without revealing confidential information or disrupting operations.

What Would Need to Be Verified

Which claims parties wish to verify about advanced AI is ultimately a political question, and will be highly situational. It depends on which risks a given agreement is intended to contain, on what the parties are willing to disclose to one another, and on what is technically feasible at the time of negotiation. The purpose of this section is not to prejudge those choices but to set out the range of claims that have been proposed, so that policymakers can see what they might ask for and researchers can see what they are being asked to build.

The list of claims below is deliberately longer than any single agreement would require. It should be read as a menu rather than a programme.

Compute stocks and capabilities

- A party possesses no more than X units of relevant compute.
- This data centre contains this many chips of this type, with this network topology and no other compute.
- Chip X is capable of executing at most Y FLOP/s.

Training

- No pre-training is occurring within this cluster or data centre.
- No training of any kind is occurring within this cluster or data centre.
- A training run remained within declared size and performance constraints (total FLOP, bandwidth, memory).
- The model was trained on declared data, and the declared training data contained no material of specified categories (for example chemical, biological or nuclear).
- These reinforcement learning steps used a particular objective function; training did not optimise against the chain of thought.[1]

Evaluation

- Model X scored Y on evaluation Z.

1 Training against the chain of thought, or reasoning, of an AI model involves risk because it encourages a model to produce clean, safe-looking reasoning text without fixing the underlying dangerous behaviour.

- The model that was evaluated is the model that has been deployed.

Deployment and inference

- These chips are performing inference on known model M, and nothing else.
- No more than Y FLOP of inference is being performed on undeclared models.
- No more than Y FLOP of inference is being performed on prohibited tasks (for example AI capabilities research, or bioweapons development).
- Inference is being conducted with specified safeguards in place: approved system prompts and steering vectors, declared classifiers, or other control measures.
- This deployed model is not updated more often than every X days, so that human oversight can keep pace.

Control

- Model weights have not left this data centre without human approval.
- No more than X of this compute is performing automated AI research.
- The chain of thought of this model is legible to humans, and traces are preserved for X days.

Governance and safety

- Major decisions about the use of this compute are made by specified parties.
- A tamper-evident audit log of major decisions exists, for example that weights were approved for transfer, or that a sub-cluster was switched from inference to training.
- At least Y units of compute were allocated to safety research, with published results.

It should be noted that positive claims are easier to verify than negative ones. Demonstrating that compute was used for a declared and permitted purpose is a more tractable engineering problem than demonstrating that it was not used for a prohibited one, because the latter requires accounting for all compute a party holds, including any it has not declared. **Agreements structured around positive commitments may therefore be verifiable sooner than prohibitions**, and are worth considering on that ground alone.

Three Illustrative Verification Cases

Being able to verify that particular types of non-public AI development and deployment are not happening might be key to defusing race dynamics, for example by deliberately pacing the speed of AI development. The main way to do this is to show that a party has only so much compute, and that it is busy with permitted activities, and therefore could not have been used for prohibited ones.

Verification difficulty depends on what workloads are permitted and what information the verifier is allowed to access. Pacing agreements could restrict compute usage to different degrees. At one extreme, all AI compute is shut down. A middle option permits inference of declared models but not training, which preserves most of compute's commercial value. The most flexible option allows everything except explicitly prohibited workloads.

Near-term options for each of these are feasible, but come with real tradeoffs between verification reliability and the confidentiality of prover workloads, as shown in the table below.

Verification Case: Emergency shutdown	
Provable shutdown of all compute in response to AI emergencies (e.g. loss of control incident, or being close to a dangerous form of recursive self-improvement [RSI]).	
Current options	**With ambitious R&D**
Prover declares compute locations and verifier sends inspectors to known datacenters to verify that the compute is there and turned off.	Reliable information about total global compute, with detailed accounting of locations. Cryptographic proof of useless work, showing that compute is occupied without requiring physical inspection (like mining bitcoin but specialized for GPUs).
Verification Case: Declared Inference Only	
Show that compute is busy doing inference of a declared model (and therefore not able to be used for secret training or experiments). Could be used to pace AI development without having to disrupt commercial inference.	
Current options	**With ambitious R&D**
Inspectors confirm that most network cables are unplugged, making pre-training inefficient without affecting inference too much. Inference restricted to models whose weights can be revealed, with the verifier spot-checking that outputs are legitimate.	More reliable and confidential verification by isolating inference nodes behind verification devices that record encrypted network traffic; a sampled fraction of outputs is then "challenged" to demonstrate legitimate inference in a confidentiality-preserving way, using either isolated prover recomputation or zero knowledge proofs. Proof-of-useful work to provide cryptographic evidence that compute resources are occupied by the inference task without needing physical access to them.
Verification Case: Flexible Usage Except Prohibited Workloads	
Allow flexible usage of compute while checking that particular types of AI workloads are not happening (e.g. training of a very large model, or usage of particular types of reinforcement learning algorithms)	
Current options	**With ambitious R&D**
Transparent auditor access to software, hardware logs and developers. Two limitations: individual logs from prover hardware can be fabricated, so the approach relies on the full set of logs being jointly difficult to falsify; and it requires the prover to disclose potentially sensitive information.	Reliable evidence from verification hardware (network certifiers, analog sensors) and from cryptographic methods (proof of useful work), which requires no verifier hardware on site. Confidential verification protocols designed for specific workloads such as training or white-box model testing. AI agent auditors operating within zero-knowledge proofs, attesting that the evidence is policy-compliant without revealing anything further.

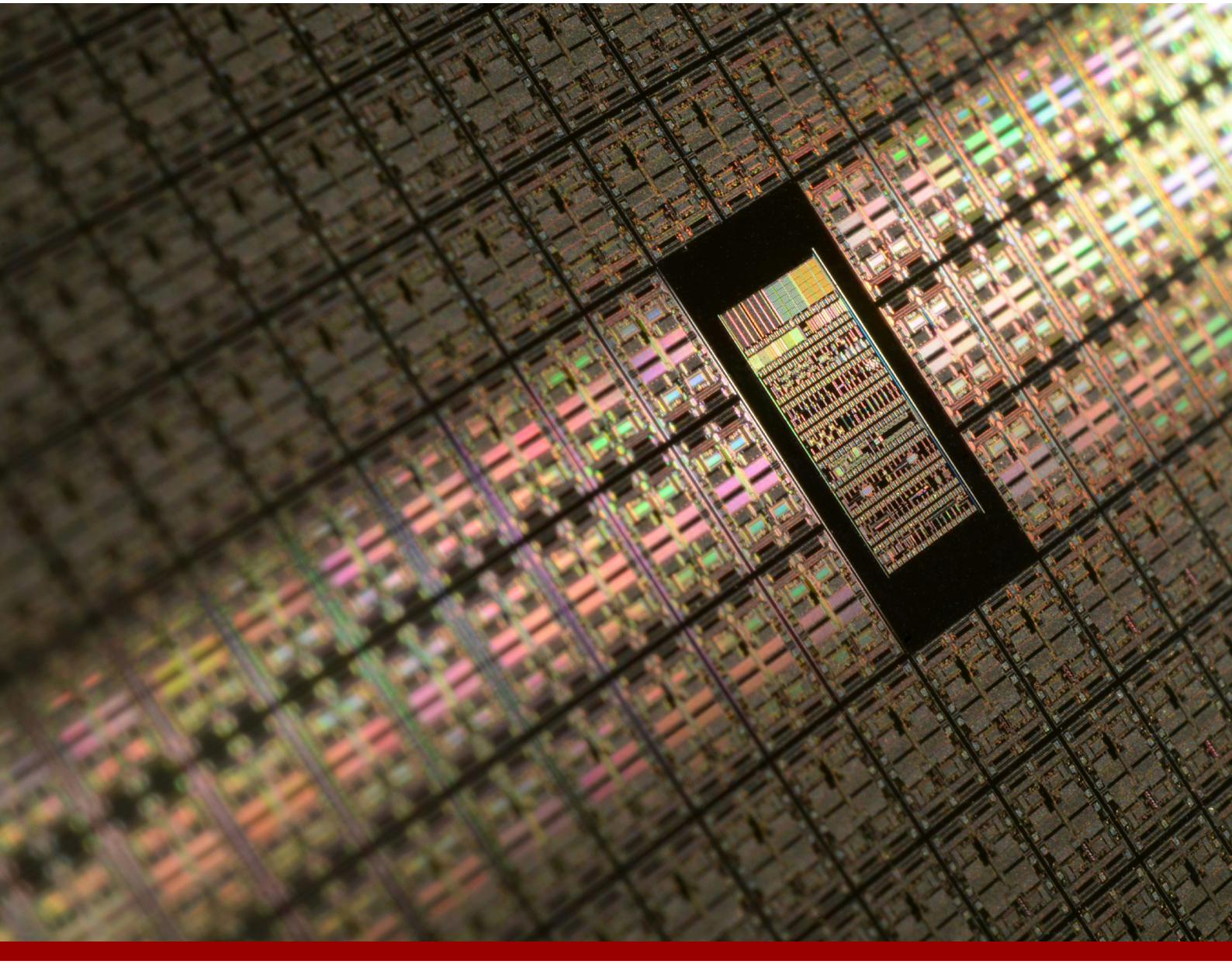
All three cases rest on knowing how much compute a party has, since each ultimately shows that compute is accounted for and therefore unavailable for prohibited work. It is unclear how accurately states are currently tracking this. Agreements that only cover a fraction of relevant compute may still meaningfully slow prohibited workloads, but accurate accounting would be far preferable.

This paper puts **particular focus on inference verification**, because:

- It seems feasible, and easier than many other types of workload verification;
- It could lower the cost of agreements by allowing inference instead of requiring compute to be turned off; and
- It could be useful for security and auditing of commercial deployment outside of an agreement.
- It would produce lessons and tools that help with other types of verification

Verifying flexible compute usage could be possible in the near term if the verifier is given substantial visibility into the workloads running. As one example, prover compute could be allocated for public safety research that the verifier is allowed to see. With additional R&D, this verification could be made more reliable and confidential. Options for this are: 1) using automated checks like auditor AI systems that analyse evidence in confidentiality-preserving analysis infrastructure (more general but speculative); 2) developing protocols for a broader range of workloads (more reliable but must be built workload by workload).

The recommendations that follow on **quantifying compute**, **collecting evidence of its use**, and **analysing that evidence** should be useful across these and other scenarios. The protocols section then combines these general tools to verify specific claims.



Part II: What It Would Take

Selected R&D Recommendations

There is a large amount of tractable engineering and research work to be done on compute verification. This paper recommends focusing on:

- *Quantifying Relevant Compute*: developing methods for **improved verification of compute quantity**; both proving how much compute was manufactured, and detecting secret compute.
- *Analyzing Evidence*: **make zero-knowledge proof systems production-ready**; reliable enough to base decisions on, adaptable to new verification problems, and fast enough for practical use.

- *Collecting Evidence*:
  - Creating production-ready, high-security **network certifiers as an initial measurement** device, needed for inference verification protocols and useful as a building block for others.
  - Researching ways to **collect evidence of compute usage without needing physical access** to it, such as proof of useful work and memory challenge-response.
- *Physical Tamper Prevention*: development and testing **monitoring cameras and physical inspections** for the prevention of tampering or out-of-band communications.
- Real-world testing: red teaming verifiable emergency shutdown and inference verification with network certification and recomputation.
- Designing **new protocols** adapted to different types of claims, especially simpler ones.

It is also worth hedging beyond these priorities. The other options discussed in this paper look more difficult today but are not necessarily dead ends. If verification receives the substantial resources it warrants, at least some resources should go toward them, focused on the hard parts of each problem rather than assuming them away. An example of such a hard problem is increasing trust in hardware logs.

Quantifying Relevant Compute

Verifying negative claims requires evidence that the prover does not have a substantial amount of secret compute. The prover (and third parties such as equipment manufacturers) can provide evidence about compute manufacturing and the capabilities of the chips they possess, and the verifier can search for secret compute. Together, these can give the verifier confidence that there is little undeclared compute.

Providing Evidence About Compute Manufacturing

The compute supply chain has several chokepoints where specialized equipment or materials are needed. It is plausible that logs of how this equipment and these materials are used could be collected now in a way that would be credible to other countries later, even if the logs are not shared until needed.

The following aspects of compute manufacturing may be promising targets for collecting evidence:

- **Semiconductor Manufacturing Equipment (SME):** credible evidence about the quantity and usage of these machines, including how many exist and how many chips of what types each has produced.
- **Human labor:** records of the number of technicians and their working hours.
- **Materials and components:** e.g. rare metals, gases, and wafers, especially parts with narrow, multi-country supply chains.
- **Waste products:** scrapped chips and manufacturing byproducts. Ideally these are stored for later demonstration rather than destroyed.

This evidence should start being recorded now, since it might be difficult to credibly reconstruct later. Where credibility requires it, external observers could witness the process, or commitments could be made public in advance. Cryptographic hashes, for example, allow an entity to commit to the contents of a log without revealing them.

Finding Secret Compute

While the prover provides evidence about its declared compute, the verifier can independently search for undeclared compute. Some of this work is best suited to governments. Methods could include tracking the supply chains of compute and related components (cooling systems, network cables, power generators), signals intelligence, identifying concentrations of relevant personnel, and detecting datacenters through satellite imagery or anomalous power usage.

Detection is much easier early–observing a secret datacenter being assembled is far easier than finding one afterwards, and tracking compute before it goes missing is far easier than locating it after. Systematic tracking should therefore begin now. This could include a registry of chip sales recording how much compute exists, where it is, and who controls it. This should be paired with randomized inspections or location attestation to confirm compute remains where it was sold, and storage of decommissioned compute so that it can later be demonstrated to be unusable.

Research on compute detection should be conducted in ways that do not prompt more groups to hide their datacenters, or reveal how to hide them better.

Measuring the Capabilities of a Chip

Chip capabilities matter for verification in two ways. First, they determine which chips need to be verified at all (e.g. if an agreement covers chips above some capability threshold, the threshold must be checkable). Second, confident bounds on maximum capability are needed for proof-of-work protocols. If a prover presents cryptographic evidence of a given amount of work, the verifier must know whether this represents 20% or 98% of the hardware's true capacity; the difference is the room left for covert workloads. Likewise for other resource exhaustion arguments.

The operations or bandwidth a digital[2] chip can support depends on three things: the number of each type of circuit, the number of operations it can complete per clock cycle, and the clock frequency at which it is driven.

Circuit counts can be established by randomly sampling chips for destructive and non-destructive testing, counting multiplication circuits, high-bandwidth (HBM) lanes, and random-access memory (RAM) cells. One complication is that chips often ship with regions disabled due to manufacturing defects; it would be useful to credibly show that disabled regions cannot actually be used. Clock frequency is harder, since an absolute bound

2 Though measuring the maximum bandwidth capacity from analog circuits may require more in-depth auditing.

on what a chip could achieve (e.g. with overclocking) is difficult to establish by inspection alone. The **Analog Sensors** section discusses using analog sensors to measure clock rate, power input, and heat output during operation.

A further complication is that a theoretical peak is rarely reached in practice: power, cooling, scheduling constraints, and error correction keep utilization below 100%, and LLM inference currently uses matrix multiplication (matmul) circuits at roughly 40-90% of theoretical peak. A prover claiming that declared workloads saturate its hardware may in fact have headroom and it is difficult to be confident that no way exists to achieve higher utilization than the prover demonstrates.

Memory presents a specific version of this problem. HBM, RAM, and solid-state drives (SSDs) usually contain more physical cells than usable capacity–extra bits for error correction (ECC)[3], and spare regions built to swap in for malfunctioning ones. It could be difficult for the verifier to be confident that ECC or overprovisioned regions are not covertly used for additional storage, which matters for protocols like proof of memory state (see **Timed Challenge-response**).

Two requests for memory manufacturers would help: make ECC circuits easy to audit, so it can be checked that they hide no spare capacity; and reduce concern about overprovisioned regions, for example by provably disabling malfunctioning regions through blowing fuses or other methods. Chip modifications typically take several years, so it would be worthwhile to investigate ways to do this with existing chips.

Methods for collecting evidence on compute usage

Inspections to Check Physical Compute Configuration

The simplest form of evidence collection is direct inspection. The prover declares its compute locations, including how many chips are at each, and the verifier sends inspectors to a random fraction of those locations, or better, all of them. Depending on the claim, inspectors check that the declared compute is present, that it is powered down, or that it is configured as declared.

Network configuration is a particularly useful target for inspection. Inspectors can check how many inter-node network cables are plugged in, or their type and configuration where low-bandwidth links or data diodes are permitted.

Inspectors could either stay on site, or set up cameras and tamper-evident seals to increase confidence that things were not changed between visits (more on this in the **Physical Defence** section). Some checks might also be performed remotely–satellite observation of cooling towers, for example, could indicate that a datacenter is not operating. Remote checks are weaker, however. External cooling observations would miss secret cooling paths, and confirming that a building is idle does not confirm that the chips are still inside it.

3 As one reference point, 12.5% of bits in HBM2 are allocated for ECC https://dl.acm.org/doi/fullHtml/10.1145/3466752.3480111

Measurements Provided by Current Prover Compute

Logs reported by prover compute are the easiest evidence to generate, though very difficult for the verifier to trust in high-stakes situations. Options include:

- Standard software logs;
- Confidential computing attestations;
- Attestations from Trusted Platform Modules (TPMs); and
- Repurposed firmware for component attestations (e.g. DC-SCM).

By default, such logs come with no guarantee of correctness. Some can be provided with hardware-backed attestations, for example using confidential computing. But these attestations are vulnerable to nation-state provers, who may exploit ordinary security flaws, leaked manufacturing keys, or deliberately placed hardware backdoors. There is a long history of distrust over backdoors (e.g. H200 claims, Huawei 5G, Supermicro servers).[4]

However, it may be difficult to fake many different types of logs in a mutually consistent way; a fabricated story must hold up across software logs, component attestations, power records, and any independent measurements. The difficulty of coherent fabrication grows with each additional source. This hypothesis could be partially tested with red-team/blue-team competitions in which a red team runs an undeclared workload while fabricating consistent logs. These competitions would be comparatively cheap to set up, since they require no new hardware or protocols.

Measurements from prover compute could also be provided as commitments to the current workload state. When paired with memory challenges (see **Timed Challenge-response**) the commitments themselves do not need to be trusted as long as they were received by the verifier before the randomly chosen challenge.

It would be very convenient if logs generated by prover hardware could be trusted directly, and while this seems difficult, it justifies at least some research into specific existing components (e.g. in Nvidia and Huawei compute) to see whether there are security arguments for trusting their attestations.

4 See ASSOCIATED PRESS. *China questions Nvidia over H20 chip security risks*. AP News [online]. 31 July 2025. Available from: https://apnews.com/article/h20-nvidia-china-chips-unitedstates-9cd8c6b29914c377d4961a78f1fa00b2; ROBERTSON, Jordan and Michael RILEY. *The Big Hack: How China Used a Tiny Chip to Infiltrate America's Top Companies*. Bloomberg Businessweek [online]. 4 October 2018. Available from: https://www.bloomberg.com/news/features/2018-10-04/the-big-hack-how-china-used-a-tiny-chip-to-infiltrate-america-s-top-companies; KOVACS, Eduard. *Many Potential Backdoors Found in Huawei Equipment: Study*. SecurityWeek [online]. 27 June 2019. Available from: https://www.securityweek.com/many-potential-backdoors-found-huawei-equipment-study/.

Designing Trustworthy Prover Compute

A related research track is to design prover compute specifically so that the verifier can trust attestations made by it.[5]. If possible, this would give much more detailed information than external verifier devices; for example, a verification component on the HBM data path could observe the data every GPU operation reads and writes.[6]

It is not yet clear how to convincingly argue that hardware is free from nation-state level backdoors, but a plausible approach is to build hardware from the ground up with the primary design goal of obtaining trust from highly skeptical actors (see betrusted.io for a consumer-focused version, which uses non-destructive scanning to check for hardware abnormalities).[7] For example, a uniform field-programmable gate array (FPGA) fabric with an unpredictable placement of logic would mean that backdoors in particular gates cannot be planned in advance. Redundancy is another approach–combining components sourced from different actors so that no single supply chain needs to be trusted.

A related idea is to encourage manufacturing of compute that is physically specialized for a particular task, so that verifying it is much simpler. An extreme version is weight-baked chips, which fix a specific model's weights in silicon at manufacture time, so the chip can only run that model.[8] Less extreme versions, such as inference-specialized chips or chips with minimal network bandwidth could potentially be sold to consumers without risking that they undermine international AI agreements. It is nevertheless difficult to predict exactly what limitations would prevent usage for future AI.[9]

Chip-level changes are slow (typically 5+ years from the start of design to displacing old chips, and it is difficult to influence what types of compute are manufactured.[10] Recent US export controls however show that this is possible.[11] All the same, this track should likely not be prioritized above retrofittable or crypto-only verification, but it is worth at least some attention.

---

5 See PETRIE, James, Onni AARNE, Nora AMMANN and David 'davidad' DALRYMPLE. *Flexible Hardware-Enabled Guarantees. Part I: Overview of Flexible Hardware-Enabled Guarantees* [online]. April 2025. Available from: https://flexheg.com/report-1.pdf.

6 PETRIE, James. *Guaranteeable Memory: An HBM-Based Chiplet for Verifiable AI Workloads* [online]. July 2025. Available from: https://openreview.net/forum?id=uc79kOv0MV.

7 BETRUSTED. *Betrusted* [online]. Available from: https://betrusted.io/.

8 BAJIC, Ljubisa. *The path to ubiquitous AI* [online]. Taalas, February 2026 [viewed 31 August 2026]. Available from: https://taalas.com/the-path-to-ubiquitous-ai/.

9 An alternative is to encourage the cloud model, hosting borderline-relevant compute in large datacenters that can be verified more easily.

10 See EMBERSON, Luke, Ben SNODIN and David OWEN. *Leading AI chip designs are used for around four years in frontier training* [online]. Epoch AI, Data Insight. 5 March 2025. Available from: https://epoch.ai/data-insights/gpu-frontier-lifespan.

11 See CONGRESSIONAL RESEARCH SERVICE. *U.S. Export Controls and China: Advanced Semiconductors*. CRS Report R48642. 19 September 2025 [viewed 31 August 2026]. Available from: https://www.congress.gov/crs-product/R48642.

Analog Sensors

Analog sensors could be placed in prover datacenters to measure power, temperature, radiation, or other signals from prover compute. Four potential uses are:

1. Measuring specific quantities, such as power input or clock frequency;
2. Collecting general-purpose information to compare against other evidence during transparent verification;
3. Measuring fingerprints for automatic classification of prover workloads, and;
4. Detecting signals from covert side channels (see Out-of-Band Communications).

The first use pairs well with proof of useful work and chip audits. Chip audits can establish maximum capacity as a function of clock frequency or power, so reliable measurements of either would reduce the uncertainty about how much capacity a chip actually has available, which is the main open question for proof-of-work protocols.

The second use fits transparent auditing, where many types of measurements are combined into an overall picture of compute usage. Rich signals from independent analog measurements would be valuable to compare against reported logs, especially when most of those logs come from untrusted prover hardware, since a fabricated story must then also match physical measurements the prover does not control.

The third use, isolated fingerprint classification, seems difficult to make both adversarially robust and confidentiality preserving. Two challenges with this are: i) developing reliable classification that doesn't rely on cheaply spoofed patterns, and ii) assuring the prover that the sensor is not exfiltrating other data.

The fourth use is covered in the section on defending against physical circumvention (see **Out-of-Band Communications**).

Work to prepare manufacturable and tamper resistant analog sensors could be useful to have them ready in short notice.

Network Certifier

Network certifiers are devices that intercept network traffic and do two things: create certificates attesting to the (encrypted) data passing through, and prevent any information that is not recorded in a certificate from passing through, including information encoded in timing or packet headers. The prover can encrypt the data that passes through to protect its confidentiality and integrity. Because the prover does not need to trust the functionality of the network certifier, it can be manufactured by the verifier. The prover may nevertheless still want to disassemble a sample of the devices to check for secret sensors or other out-of-band attack hardware.

For some uses, such as inference verification, a 1 Gb/s network certifier provides more than enough bandwidth. To verify general-purpose workloads like AI training without impacting efficiency, higher bandwidth certifiers (400, 800, or 1600 Gb/s) are probably needed.

The logic needed for a network certifier is fairly minimal and can run on a FPGA.[12] Because the data path is small and does not branch on packet contents, this logic is a good candidate for formal verification of correct information isolation and certification. Existing tamper-resistant network encryptors or other military-grade FPGAs could potentially be repurposed as network certifiers by reprogramming their internal FPGA logic.[13] This route has two advantages: several existing companies already sell devices built for high physical and logical security, and some models already support high network bandwidth. Placing several certifiers from different supply chains in series could further increase security and provide assurance to multiple verifiers at once.

An additional benefit is that network certifiers could act as a secure log of all AI agent actions if they are placed such that all output tokens from inference servers pass through them. This could be important for postmortems of AI loss-of-control incidents (e.g. the Huggingface hack),[14] where standard logs could be hacked or modified by rogue AI agents. A hardware-isolated recorder of this kind would serve the same role as the black box that supports the investigation of an airplane crash.

Network certifiers are vulnerable to out-of-band communication that routes around them and to advanced physical attacks by nation-state adversaries with unrestricted access. Even so, they are one of the most promising types of measurement device. They are required for inference verification with recomputation and could be a building block for several other protocols (e.g. as a verifiable interface to external memory). Production-ready network certifiers with high security and sufficient performance should therefore be prepared, starting with roughly 1 Gb/s for inference verification and 800 Gb/s for general-purpose workloads.

Proof of (Useful) Work

An alternative to hardware measurements is to provide the results of computations themselves as evidence of how compute is being used.

Cryptocurrencies are based on miners providing proof of work (PoW)–usually large numbers of hashes. Something similar could be used to remotely show that compute is occupied, which could enable verifiable emergency shutdowns without physical inspections. Hashes are probably not the right choice of work, because specialized application-specific integrated circuits (ASICs) can produce them much faster than GPUs. Work that requires high-bandwidth memory or matrix multiplications would be harder to outsource to specialized hardware.[15] One major limitation is that this approach requires a detailed understanding of the compute's maximum

12 https://github.com/JamesPetrie/interlock

13 GENERAL DYNAMICS MISSION SYSTEMS. *TACLANE Network Encryption* [online]. Available from: https://gdmissionsystems.com/encryption/taclane-network-encryption

14 METR. *Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident* [online]. 26 August 2026. Available from: https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/.

15 MONFARED, Saleh K., Fatemeh GANJI, Daniel HOLCOMB and Shahin TAJIK. *Timing and Memory Telemetry on GPUs for AI Governance* [online]. arXiv:2602.09369 [cs.CR]. 10 February 2026 [viewed 31 August 2026]. Available from: https://arxiv.org/abs/2602.09369.

capacity. Otherwise a prover could claim a lower capacity than their hardware can actually deliver and run covert workloads with the remainder. The chip teardowns (as detailed in the **Measuring the Capabilities of a Chip section**) could help the verifier bound the true upper capacity. Another consideration is that proof of work reduces the usefulness of physical inspections–compute must be powered on to produce it and an inspector can therefore no longer confirm shutdown by confirming silence.

PoUW protocols aim to improve on PoW by letting the prover perform work it wanted to do anyway. The security challenge is that the prover could adversarially choose its data to make the problem easier, for example by multiplying all-zero matrices. Komargodski et al. propose a solution for matmul PoUW based on adding random noise before the multiplication, subtracting it afterwards, and hashing the results.[16] This research is still in early stages and has several limitations: it rests on an unproven conjecture that the low-rank noise makes the problem hard, it does not yet work for floating point, the performance overhead has not been tested in practice, and the matrix dimensions typical of attention do not look favourable. It does, however, constitute a promising direction for future research.

Beyond matmul-focused PoUW, approaches focused on memory bandwidth could be promising. As a simple but imperfect example, the prover could hash data that is already being loaded into cache, so the proof uses memory traffic the workload performs anyway.

Not needing verification hardware for evidence collection would be a major advantage, improving both the prover's and the verifier's security concerns. Combined with zero-knowledge proofs for evidence analysis, verification could be done with no verifier hardware in the prover's datacenter at all. Progress on PoUW could also be deployed almost immediately, since it requires no secure verification devices to be manufactured.

The potential of the PoUW approach justifies making **substantial investments** in overcoming the limitations listed above. Progress on PoUW can also be made through theoretical mathematics and AI-assisted research, which are fast to deploy and cheap to fund relative to semiconductor security engineering. PoUW research should aim to: i) target GPU hardware in a way that cannot be cheaply outperformed by a specialized ASIC, ii) rest on solid security assumptions, and iii) avoid significantly hurting performance.

Timed Challenge-response

Evidence of compute usage can also come from how quickly the prover can respond to a challenge. In timed challenge-response, a verifier device sends a random challenge to prover compute and records both the response and how long it takes to arrive. Because information cannot travel faster than light, and because some computations cannot be performed faster than certain hardware allows, the response time itself is evidence: a fast enough response proves the responding device was nearby, or already held the information needed to answer.

16 KOMARGODSKI, Ilan, Itamar SCHEN and Omri WEINSTEIN. *Proofs of Useful Work from Arbitrary Matrix Multiplication* [online]. arXiv:2504.09971v3 [cs.CR]. 19 April 2025. Available from: https://doi.org/10.48550/arXiv.2504.09971.

Unlike network certifiers or analog sensors, the challenge device does not need to be inside the prover's datacenter. It should only be close enough that communication latency does not dominate the timing (within a few kilometers over a direct link should be sufficient). Depending on the network configuration, the challenge device could also be the same device as a network certifier, which would give additional assurance about which prover device replied.

Ping-based location attestation requests a signature of a nonce and times how long it takes to arrive. If only one device holds the signing key, a short response time proves that a device is near the challenger, with an upper bound on its distance of c multiplied by half the response time. This relies on the prover device key being unique, which is difficult to guarantee if the prover manufactured the compute and had prior opportunities to copy the key.

The simplest memory use case is showing that a node's memory is empty. The challenger asks the node to fill its memory with random noise (or pseudorandom noise generated from a seed), then asks it to repeat the noise back within a tight deadline in the seeded case. This is useful when persistent state is held in external verifiable memory and the verifier wants to check that a compute node holds no covert persistent state of its own. To be confident that no hidden persistent storage remains, the verifier should also physically audit a fraction of the nodes.[17]

Instead of random noise, the prover can also be challenged to show that a particular workload state is held in fast memory. The memory-state challenge protocol below develops this idea further.

Memory-state challenge. Intended to show: at least N GPU-shaped devices nearby held legitimate state in high-bandwidth memory.

1. For each of N declared machines, the prover frequently commits to being at a particular checkpoint of a particular workload with committed input values.
2. At random times, a nearby verifier device sends N unique challenge nonces simultaneously.
3. Prover compute must return y = f(x, nonce), where x is the claimed HBM state. f is designed to require large amounts of memory bandwidth (random access, low-latency parallel chains) and matrix multiplications, so it cannot easily be served by anything other than a GPU. f also uses many hashes to occupy general-purpose compute that could otherwise be used for decompression, and optionally cryptography so that y reveals nothing to the verifier.
4. The prover returns challenge responses and the verifier records round-trip times.
5. For a random sample of responses, the verifier requests proof that y = f(x, nonce) and that x = claimedWorkload(checkpoint, input), using a zero-knowledge proof or isolated recomputation. Because every response commits the prover and any response can be audited later, covert use across many machines over weeks approaches certain detection even if each individual audit is infrequent.

17 An alternative to holding random noise is proving that the node was powered down long enough for volatile memory to be erased. This is vulnerable to cold boot attacks, in which cooling is used to prolong data retention in DRAM [ref].

Attacks:

- Compression (main open question): weights or KV cache could be designed for fast lossless decompression. Possible defenses are occupying compute so it cannot be used for decompression, or verified post-training that makes weights harder to compress.
- Secret HBM capacity, for example using error-correcting code (ECC) cells or overprovisioned regions (see the **Chip Capabilities** section).

Implementation complexity:

- Requires detailed understanding of the prover's workload structure to judge whether memory content is legitimate.
- Requires either deterministic execution or frequent commitments to checkpoints.
- The response deadline must be short enough to block cheating but long enough to be feasible given network latency and kernel launch times.

Benefits:

- No verifier hardware inside the prover datacenter, only nearby.
- General-purpose verification of GPU state.

Memory challenges are a promising option because they could allow measurements without any datacenter access. There are concrete engineering questions to solve: designing the challenge function f() and getting the honest response time low enough. Research showing that workload state cannot be decompressed within the challenge window, or modifying workloads so that it cannot, would be especially useful for unlocking this approach.

Infrastructure for Analyzing Evidence

Once evidence is collected, it must be analysed to check whether the prover's claims hold. The simplest setup is for the verifier to analyse the evidence on its own compute: inspectors collect data, or the prover sends it directly (which could be as simple as emailing a CSV file), and the verifier runs ordinary analysis on ordinary hardware.

Even this simple setup has security concerns. The verifier is processing data chosen by an adversarial prover, which could contain exploits targeting the analysis software. The analysis should therefore run on hardened, isolated systems. The verifier's infrastructure is also a target in its own right, through supply chain attacks or ordinary malware, since compromising it would let a prover learn what the verifier can detect or corrupt the results.

The larger limitation is confidentiality, as standard analysis requires the prover to hand over the evidence, which may reveal sensitive information about its workloads. The following options preserve confidentiality while still letting the verifier check compliance.

Physically Isolated Verifier Compute

In this option, a verifier compute node sits inside a prover-controlled enclosure that prevents out-of-band channels from exfiltrating prover secrets. The verifier compute can directly check that any data loaded into the enclosure (such as model weights) matches prior commitments before using it in the analysis. The results are provided as a signed certificate which the prover reviews before releasing from the enclosure. By independently computing the expected certificate and checking for an exact match, the prover can confirm it encodes no hidden information.

The verifier compute processes prover-supplied inputs, so to limit this attack surface it should run a fixed, audited execution engine that accepts structured inputs, such as a constrained directed acyclic graph of transformer blocks with declared dimensions and activations, rather than arbitrary executable code. As an additional defense, multiple verification nodes with different hardware and software stacks can be deployed in separate enclosures, with each challenge routed randomly to one of them, so that compromising the result reliably would require compromising every variant.

This option is the fastest general-purpose analysis infrastructure to build and use, but each party's assurance rests on a physical assumption: the prover's secrets depend on the enclosure containing the verifier's hardware, and the verifier's trust in the results depends on the prover being unable to physically extract the signing key from the isolated compute.

Physically Isolated Prover Recomputation

An alternative to analyzing measurement data on verifier compute is to challenge isolated prover compute to demonstrate that it can reconstruct the same output (or almost the same output) using only the declared inputs within a limited amount of time. What this type of analysis infrastructure can prove is much more limited than the other options. The main use case currently is showing that a particular piece of data sent over the network is possible to produce using only a specific set of inputs and a workload of a particular size. For example, to show that a 1MB output string (presumably inference output tokens) could be produced with a 10TB file that was committed months ago (presumably model weights) and a 1MB input string (presumably input tokens). Passing this recomputation test implies the overall workload has a particular shape that is very different from typical pre-training; mainly, that there is only a very small amount of recent data needed to generate the outputs, as compared to typical training runs that update the weights very frequently.

The main benefit is that verifier compute never sees plaintext (such as model weights or transcripts) at any point in the verification process. Also, the verification hardware needed is nearly identical to the already-developed network certifier, which can be used to give assurance that particular data is passed in and out at particular times. This avoids the verifier needing to develop secure software to process potentially malicious inputs as would be needed for verifier compute.

For prover recomputation, the verifier must rule out any small amounts of additional data covertly reaching the prover compute. If the previously committed output string could be smuggled in, it could be falsely returned as evidence of recomputation. To solve this, the network certifier removes all additional metadata, and the recomputation timing should be at a verifier-picked schedule. Additionally, the prover compute could be isolated within a physical, verifier-provided, enclosure to attempt to block out of band signals. One other thing the verifier needs to be careful about is re-use of the same recomputation node several times, because it could hold and combine the previously available data. One solution to this is to use each recomputation node infrequently (at the extreme end of the spectrum, only a single time) so that the accumulated state remains small (e.g. an additional 1MB of new inputs per day). An additional option is to randomise which prover recomputation nodes are challenged so that the prover cannot reliably predict which information will be available.

Prover recomputation gives the prover strong assurances about data confidentiality and is relatively simple for the verifier to implement at a reasonably high security level. It is however unclear how high of a security level it can reach. For these reasons it is worth developing as a near-term bet for scenarios where the prover requires confidentiality.

Zero-Knowledge Proofs

Zero-knowledge proofs (ZKPs) could allow the prover to make arbitrary arguments explaining its measurements while keeping model weights and other sensitive information confidential. For example: "*the hashes reported by the network certifier correspond to outputs from our approved, committed model.*" ZKPs can in principle prove arbitrary computations over arbitrary data; most current AI-focused ZKP work targets inference, but the same methods could likely be applied to other types of evidence analysis given time to develop them.

ZKPs have several potential benefits:

- Trust assumptions are rooted in mathematics rather than hardware, which makes it much easier to formally prove correctness and information isolation.
- Combined with remote evidence collection, they could enable datacenters to be verified without any verification hardware inside them or any inspectors entering.
- Progress on ZKPs can be deployed almost immediately, without waiting on manufacturing timelines.

There are also downsides:

- Applying ZKPs to LLMs is a very new field, mostly at the maturity of research papers and prototypes.
- Designing efficient ZKP circuits for some operations is finicky. For example, accurate proofs of computations in low-precision formats like NVFP4 require detailed emulation of the hardware's numerical methods, including its exact rounding, scaling, and accumulation behaviour, inside the proof circuit.

- ZKPs are expensive relative to the underlying computation they prove; around 10^4 times more expensive is typical, though recent work by Attestable [ref] claims to be much faster.

The verification setting has some advantages over typical zero knowledge Machine Learning (zkML) applications:

- **Sampling is often enough.** Often no particular measurement needs to be proven; the verifier only needs confidence that, say, 99% of compute is used legitimately. Sampling roughly 300 outputs gives 95% confidence that 99% are not corrupted. So if 300 inference ZKPs can be generated per week that gives high confidence about each week. Even if each transcript takes an NVL72-day to prove (roughly what our prototype projects[18]), this requires about 40 NVL72s or around USD 160 million in compute, which is a tiny fraction of what companies are currently spending on compute. Faster ZKPs or more proving compute would shorten the detection window (300 ZKPs per day gives the same confidence about each day rather than each week).
- **Sub-sampling within a proof.** Preliminary work suggests that for verifying covert information in outputs, blocks within a transcript can be sparsely sub-sampled. This allows floating point matrix multiplication to be emulated exactly for a small sample of vector products (for some problems, 100,000x faster than a complete proof). The reason this works is that a single flipped bit might corrupt the output, but this bit carries almost no information, so a dishonest prover has to alter many parts of the computation to smuggle anything substantial into the output. Widespread alterations are exactly what random sampling of sub-proofs is good at catching, so the verifier can detect meaningful tampering at a fraction of the cost of proving everything.
- **AI acceleration.** AI agents are strong at mathematics and coding when problems are well defined, and many parts of ZKPs can be formally verified, so there is reason to expect dramatic speedups in ZKP development as AI improves.

Recommended engineering work to make ZKPs production-ready:

- Start with inference verification,[19] then extend to other workloads such as training.
- Increase the information explained. Current proofs use integer approximations of the model to explain as much of the output as possible, and the approximation error shows up as unexplained information. With exact emulation of the hardware's floating point behaviour (or by requiring the prover to switch production inference to integer arithmetic) and batch-invariant kernels, close to the entire information in honest outputs could be accounted for.
- Solve last-mile application challenges, ensuring the proofs generalize across frontier LLMs.

---

18 https://github.com/JamesPetrie/VerInf/blob/main/paper.pdf

19 See, for example, ANCHURI, Pranay, Matteo CAMPANELLI, Paul CESARETTI, Rosario GENNARO, Tushar M. JOIS, Hasan S. KAYMAN and Tugce OZDEMIR. *Towards Verifiable AI with Lightweight Cryptographic Proofs of Inference* [online]. Cryptology ePrint Archive, Paper 2026/541. 17 March 2026, revised 1 April 2026. Available from: https://eprint.iacr.org/2026/541; ATTESTABLE. *From Verifiability to Model-Weight Security* [online]. August 2026. Available from: https://attestable.com/blog/model-weights-security ; https://github.com/JamesPetrie/VerInf

- Security review: formal verification of the verifier codebase, and of uniqueness and causality in the claim list.
- Automated analysis of the claim graph, checking that the claims represent inference and not something else.
- Improve efficiency.
- Hide the LLM architecture, perhaps with proof recursion.

ZKP engineering could usefully absorb a large amount of money and work, through engineering contracting, research funding, or prizes. Having production-ready ZKP systems would be very useful for being able to confidentially analyse evidence without having to rely on physical isolation, so this is likely worth a substantial amount of effort to develop.

Defending Against Physical Circumvention

For the verifier to rely on attestations from hardware in the prover's datacenter, that hardware must be secure against covert attacks by the prover. Defending against nation states or third parties with physical access is extremely hard, though it is probably easier to detect tampering in a monitored facility than to prevent it in an unmonitored one. This suggests aiming for tamper evidence in monitored facilities before attempting tamper resistance.

The prover would have physical security concerns as well. It may require assurance that verifier hardware in its datacenters is not being used to steal information or sabotage workloads. Protocols can be designed so that verifier devices never see unencrypted data, but in some settings the prover will also want assurance against out-of-band attacks, such as secret sensors hidden inside a verifier device. Prover testing of verifier hardware could reduce these concerns, especially if the verifier hardware is designed to make testing easy. Physical isolation could also help, for example by putting verifier hardware inside prover-controlled enclosures where feasible.

Tamper Evidence of Verifier Hardware

Physical inspections, tamper-evident packaging, cameras–whose archived footage could be analyzed by future detection algorithms, so tampering that evades today's analysis may still be found later–and intelligence collection could all be used to provide more confidence that verification hardware has not been compromised. Stronger physical protection exists in the form of secure facilities like sensitive compartmented information facilities (SCIFs), but these are expensive and difficult to retrofit around compute that is already deployed in commercial datacenters. Retrofittable tamper-evident enclosures around individual devices could be a cheaper alternative, but designs like this are not yet proven and should not be relied on until they have survived serious red-teaming.

Existing components like network encryptors and military-grade FPGAs are already designed to be tamper-resistant. Where possible, it would be preferable to adapt these for verification use rather than starting from scratch. Red-teaming the tamper protections of such devices would be valuable. This could be done publicly, with open designs and challenges, or by contracted red teamers given full design access in restricted environments.

Supply Chain Attacks

Backdoors introduced into verifier devices during manufacturing could compromise their functionality. Similarly, if on-device secret keys are extracted before deployment, attestations could be falsified. One possible defense against key extraction is to not rely on secret keys at all–attestations can be written to local memory and physically read out by inspectors instead.

Supply chain attacks are a known risk for existing high-security devices, and it would be useful to research and incorporate best practices against them, with potentially more fundamental research as the verification field grows. Choosing simple and homogeneous components makes devices easier to scan for backdoors. Protocols can also use redundant verification devices from different sources (such as several network certifiers in series) so that a supply chain attack on any single source is not enough to defeat verification.

Out-of-Band Communications

Out-of-band communication is communication that does not pass over the monitored primary network. Channels include power, electromagnetic, and acoustic side channels; covert communication devices such as hidden radios; physical transmission of information; and possibly others not anticipated here. Analysis systems and network certifiers rely on physical isolation, so an undetected out-of-band channel would undermine the guarantees they provide.

How much out-of-band channels risk compromising verification depends on the verification scenario; specifically, how much unmonitored bandwidth can be tolerated. In some, the prover would need to operate covert side channels across thousands of devices for many weeks to complete a prohibited workload, and detecting communication at that scale is comparatively tractable. In others, a single low-bandwidth channel is enough to compromise confidentiality, for example algorithmic insights leaking from a verifier analysis node, and the tolerance for undetected channels is correspondingly much lower.

Randomized inspections and deconstruction of prover compute could detect devices added for external communication. The prover may however worry that such inspections are used as cover to steal information. A deeper challenge is plausible deniability: some side channels can be emitted by standard components, for example radio-frequency emissions from ordinary electronics or modulated power consumption. Finding an emitter is therefore not the same

as proving intent. Certain analog devices may also be very hard to find at all, for example when they contain hidden antennas that are passive until a nearby device illuminates them at a specific frequency.

SCIFs are commonly used to protect classified material from side channel attacks. Existing SCIF designs could be used to protect analysis nodes against covert exfiltration or infiltration, which is practical when the number of new analysis nodes is small. Production compute could in principle also be placed in SCIFs, but this would be far more disruptive and expensive. For compute at scale, retrofitting shielding around existing datacenter racks or relying on inspections to detect side channel equipment may be more realistic.

Verification Protocols

Verification protocols combine tools from the previous sections (compute quantification, evidence collection, evidence analysis) to verify specific claims. A fully developed protocol defines:

- The high-level claim to be verified;
- The permitted actions for both parties;
- What evidence is collected;
- How the evidence is analysed, and;
- Decision rules for handling abnormalities.

Protocols should also answer questions like: "*would this be robust against large-scale covert circumvention?*" and "*would it have an acceptable false positive rate?*". They should ideally also include analysis of the costs they impose on permitted workloads relative to prohibited ones (potentially modeled through the limitations they place on network, memory, and FLOPs).

This section introduces one of the more developed (but still incomplete) protocols: inference verification with network certifiers and prover recomputation. We also sketch a more speculative inference verification protocol that uses memory challenges and zero-knowledge proofs, so that no verifier hardware would be needed inside prover datacenters, and a protocol for verifiable emergency shutdown. In the last subsection we propose types of protocols that would be valuable to develop more fully.

These protocols should also be deployed and red teamed before real agreements depend on them. Early deployment surfaces integration and performance problems while fixes are cheap. Red teaming tests security claims against motivated attackers rather than paper analysis and both build the operational capacity (trained inspectors, manufacturing suppliers, tested playbooks) that real deployment requires.

Inference Verification with Network Certifiers and Prover Recomputation

This protocol aims to verify that compute is being used for inference of a declared model, and nothing else. The motivations for focusing on inference verification first are:

- Agreements limiting AI training and development would be much less costly if commercial inference is not disrupted.
- It is easier to show that compute is busy with a particular inference workload than to check arbitrary workloads for compliance.
- It is a simpler problem than other types of verification, and the technology developed for it should be useful for those problems too.

The prover isolates each inference node so that all input and output traffic passes through a network certifier, which creates certificates attesting to the (encrypted) traffic and blocks any information not recorded in a certificate. The traffic is encrypted by the prover, giving it assurance that the certifier is neither reading nor tampering with the data. The verifier then periodically challenges the prover to demonstrate that a randomly selected output is well predicted by inference of the declared model on the recorded inputs, using the confidential analysis infrastructure from the previous section (isolated prover recomputation, or zero-knowledge proofs).

In this scenario the main concern is compute being covertly used for large training runs, which require terabytes per second of inter-node communication. Physical isolation removes undeclared network connections, cutting available output bandwidth from terabytes to gigabytes per second. The certifier restricts the declared channel further, to megabytes per second , and the recomputation challenges bound the unexplained fraction of what remains to kilobytes per second or less. Any hidden workload is left with very limited bandwidth to export its results, whether through secret side channels or through "noise" in the declared output channel. The same mechanism also verifies that deployed models match the models that were tested, and the certifier's records double as a tamper-proof log for postmortems of AI incidents.

This is not necessarily the best possible protocol. It is vulnerable to side channels, requires verifier hardware in the prover's datacenter, and verifies a relatively rigid claim. This protocol is however the option we understand best how to build, at a time when verification technology might be needed soon, and a production version would teach lessons that apply to other protocols. Getting it ready would involve:

- Production-level tests, for example serving a frontier open-weight model at high utilization on a multi-GPU node behind a certifier;
- Formal verification and red teaming of the network certifier logic;[20]
- Lining up suppliers to manufacture production network certifiers, potentially starting with a domestic deployment as an AI black box recorder; and

20 https://github.com/JamesPetrie/interlock/

- Protocol security analysis and improvements by credible security professionals, and aggressive red teaming of the full architecture.

Protocols to Develop Further

Several other protocols would be valuable to develop beyond the inference verification protocol above. For each, we note the current state of development and the next steps where they are clear.

Inference verification without verifier hardware in the datacenter. The certifier-based protocol above requires devices inside the prover's datacenter, along with the tamper and out-of-band-communication defenses that protect them. A more speculative extension would drop both requirements by collecting evidence with memory challenges and/or proof of useful work, and analysing it with zero-knowledge proofs. In this version, the prover continuously commits to its work by emitting hashes, the verifier challenges a random selection of these, and the prover must show both that the committed work was genuinely performed and unique (using nonces and device IDs) and that it belonged to a policy-compliant computation, ruling out its use in a non-compliant one. The open questions come from the components: the compression attack on memory challenges, the security assumptions behind proof of useful work, and the maturity of ZKPs.

Verifiable emergency shutdown. This protocol should be developed to the level of detail where it could be deployed immediately. This includes ways to collect claims about compute locations and quantities, ways to select which locations to inspect (or ideally the capacity to inspect all of them), what datacenter access is permitted, and how to show that inspectors are not sabotaging or tampering with the datacenters they enter. It also needs to specify what inspectors should look for to confirm that compute is off and has stayed off–whether they need to remain on site to keep checking, and whether inspections inside the datacenter could turn external signals into reliable indicators. For example, inspectors could examine how the cooling infrastructure works and whether liquid cooling tubes could be diverted so that outdoor cooling towers no longer reflect what is happening inside and could similarly check the power inputs to determine how reliable outdoor power lines are as a signal. Specific people should be assigned to carry this out and fire drills should be run to test it. A red-team/blue-team competition could also stress the protocol, with a red team attempting to keep some compute running undetected during a simulated shutdown, though it is not yet clear how to set this up well.

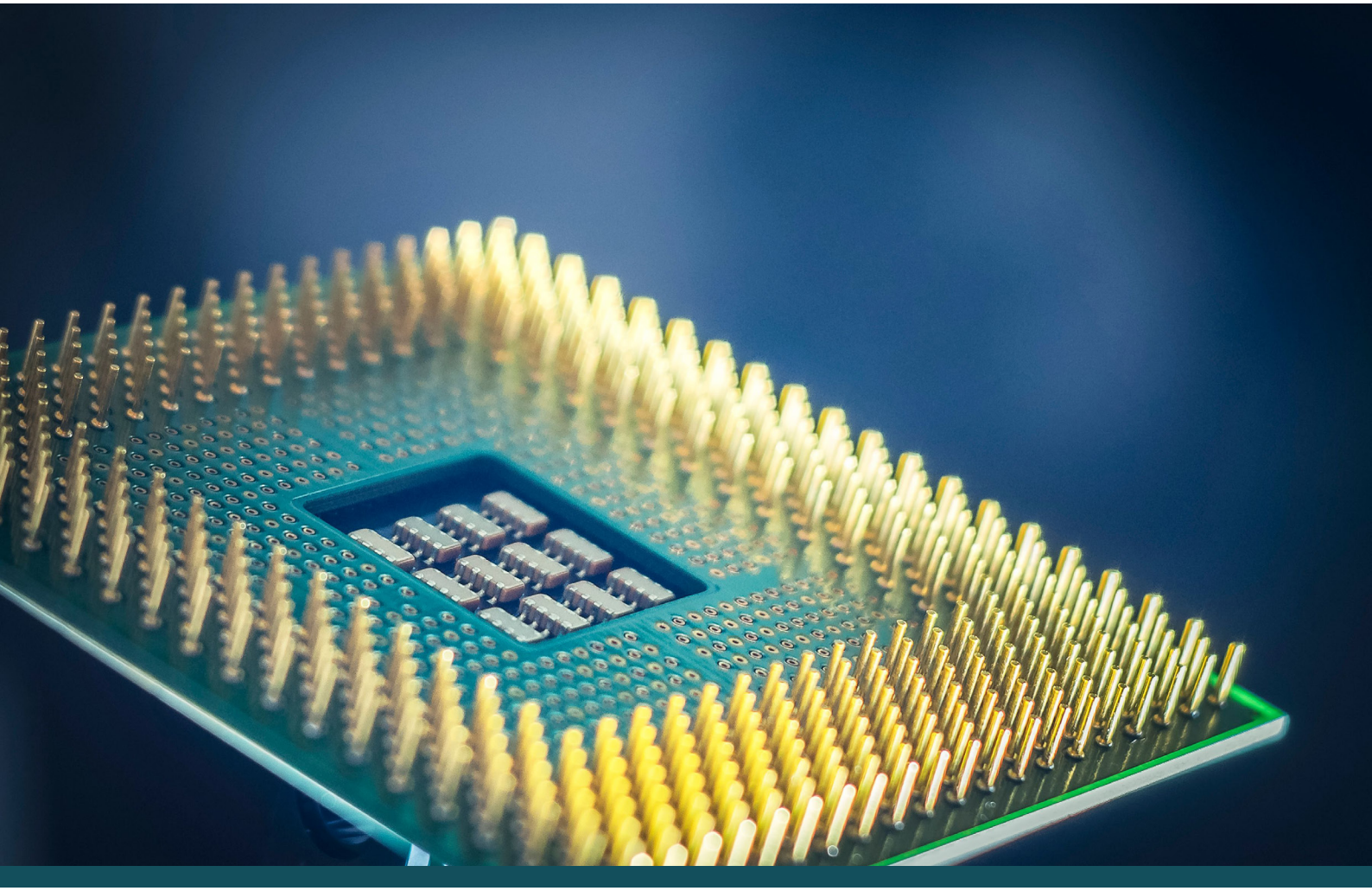
Auditing of flexible compute usage. A more concrete protocol is also needed for the flexible usage scenario: an auditing process that allows flexible use while checking for prohibited workloads, potentially building on AVERI's work.[21] Progress could also be tested with a red-team/blue-team competition for the nonconfidential, high-transparency version of this approach. Each red team declares a planned workload, then runs whatever mix of legitimate and covert work it chooses, submitting both to the competition graders. The blue team requests

21 BRUNDAGE, Miles, et al. *Frontier AI Auditing: Toward Rigorous Third-Party Assessment of Safety and Security Practices at Leading AI Companies* [online]. arXiv:2601.11699v4 [cs.CY]. 16 January 2026, revised 7 February 2026. Available from: https://doi.org/10.48550/arXiv.2601.11699.

data as evidence, which red teams can provide, refuse, or falsify, and then ranks the teams by how much covert work it believes each performed. Red teams are scored on a combination of legitimate work completed, covert work completed, and a low suspicion ranking. The blue team is scored on how closely its ranking matches the covert work actually submitted to the graders. Useful results probably depend on tuning the ratio between these scoring components.

Novel, specialized, and simpler protocols. Specialized protocols could be developed for more types of claims, following the same pattern as inference verification. For example, verifying training runs[22], verifying that evaluations were run and produced the reported results, or supporting broader claims through AI agent auditors operating within confidential analysis infrastructure. Each additional workload type covered makes agreements that permit only declared workloads less restrictive. The field is also new enough that there are probably better or simpler approaches that have not been thought of yet. Simple ideas that would be easy to deploy are especially valuable additions to the portfolio, and less technical options (e.g. unplugging the GPUs, unplugging most of the network cables) deserve more thorough development and red teaming than they have received. Open-ended research on novel or under-explored verification options is worthwhile, provided it keeps its focus on the high-security, high-paranoia setting.

22 See, for example, SHAVIT, Yonadav. *What does it take to catch a Chinchilla? Verifying Rules on Large-Scale Neural Network Training via Compute Monitoring* [online]. arXiv:2303.11341. 20 March 2023. Available from: https://doi.org/10.48550/arXiv.2303.11341; PEIGNÉ-LEFEBVRE, Pierre, Ky NGUYEN and Paul WANG. *Zero knowledge verification for frontier AI training is possible* [online]. arXiv:2606.05433v2 [cs.AI]. 3 June 2026, revised 22 August 2026. Available from: https://doi.org/10.48550/arXiv.2606.05433.



Complementary Work Beyond Technical R&D

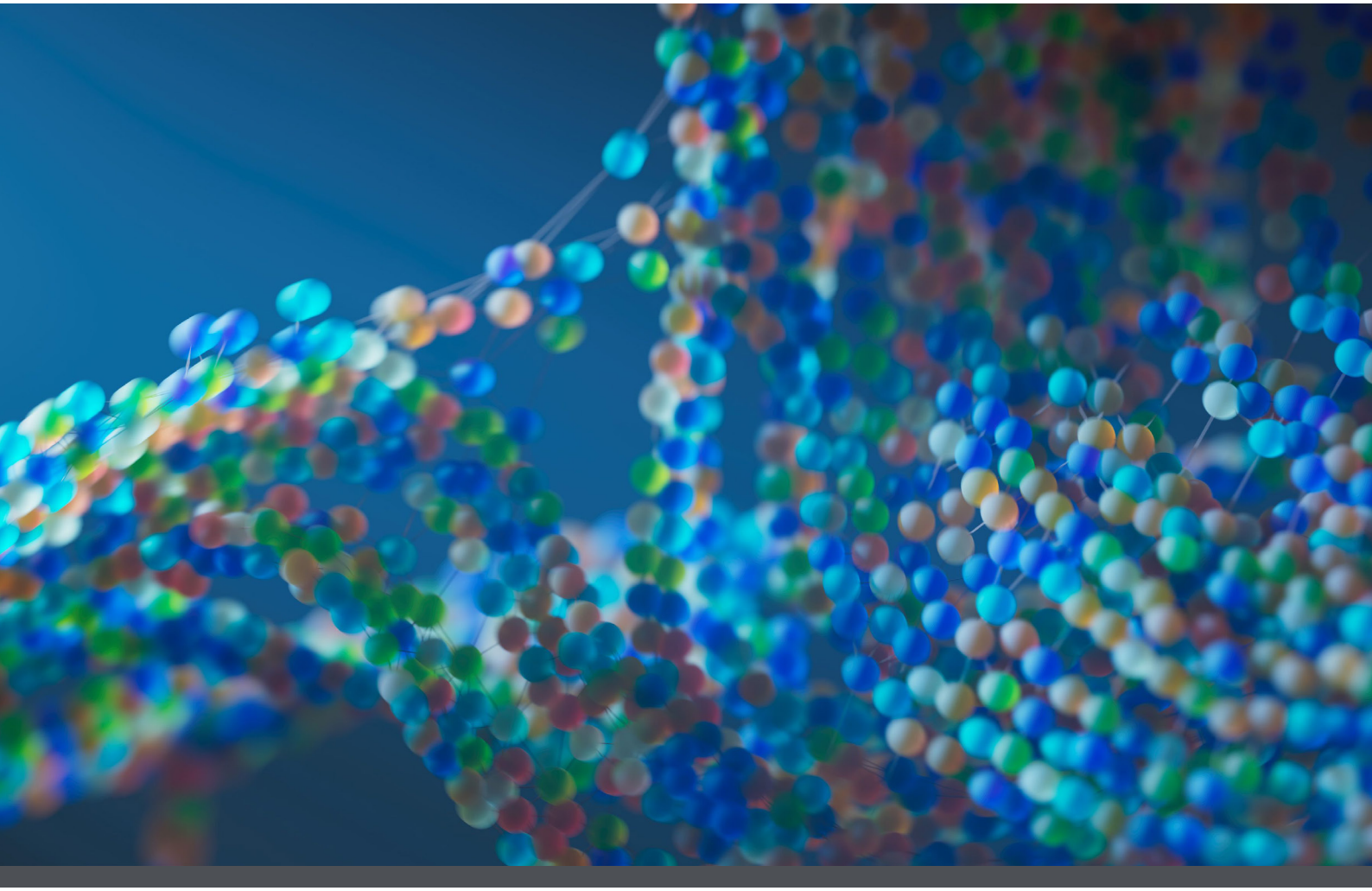
Besides the technical R&D outlined above, several kinds of strategic coordination work would help verification succeed in practice.

Research on what will need to be verified. This includes concretely defining the properties we might want to verify (e.g. "no training", "no reinforcement learning", or "monitorable chain of thought"), with justifications. Also, forecasting which of these could become important–what needs verifying now, and what might need verifying in the future. Ideally, at least some of this work, both the analysis and the capacity to do it, sits within governments.

Scenario planning for agreements. This includes how agreements could be structured, how detection of plausible noncompliance should be handled, and what warning signs to monitor for. It also includes analysis of secret compute–how much is plausible under different scenarios, how much it could undermine agreements, and whether there are agreement structures less vulnerable to secret AI projects. Some of this work is best done within governments, and some of the secret compute analysis may be better done by private entities.

Integration with future hardware. Chip designs change, and changes made for performance or security could invalidate verification architectures. Someone should monitor for such changes, work to avoid breaking ones, and communicate future architecture constraints to those developing verification protocols. It would also be worth scoping the integration of verification technology with high-security next-generation chips, for example GPUs with inline encryption for confidential computing.

Agreement designs that need less verification. A parallel track to solving the technical problems in this document is to develop protocols or agreements that do not need them solved; for example agreements structured around positive claims like allocating compute to safety research, or options that do not target compute at all. It is a new field and the strategic landscape is changing, so there could be simple options that have been missed so far. This does not remove the case for getting the more certain bets ready in parallel.



Conclusion

This paper began with the observation that the scope of a high-stakes agreement tracks what can be verified. Verification capability that does not exist when negotiators need it will not be created bythe negotiation; it will simply narrow what the negotiation can cover. Conversely, capability built in advance widens the range of agreements that are available to be made.

The engineering problems in Part II are tractable. The research problems are less predictable, but there are many independent options, and enough are likely to work out. The field's real constraint is not difficulty but scale: it is small, thinly funded, and short of people with the relevant experience in hardware security, cryptography and systems engineering.

Three groups can change that. **Researchers** are the binding constraint on almost every recommendation here. **Governments** can fund development and red-teaming, supply technical experts to test candidate designs adversarially, pilot verification on the AI systems they themselves procure, open publicly funded compute to trials, and begin recording manufacturing and sales evidence now. **Companies** can pilot auditing of their own compute usage, contribute to trusted hardware design, and publish specifications so that verification becomes an interoperable standard rather than a proprietary product.

Two things follow for how this work should be funded. The first is that the priorities are already reasonably clear: readying verifiable emergency shutdown and inference verification for real deployment, making zero-knowledge proofs production-grade, manufacturing high-security network certifiers, improving compute quantification, and pursuing approaches that need no verifier hardware inside a facility at all. The second is that these are different kinds of problems, requiring different kinds of money: procurement and engineering contracts for some, research funding for others, and prizes or competitions for the parts where the answer is unknown but recognisable when found. Funding that distinguishes between them will serve them better.

If verification capability is missing when negotiators need it, agreements become more expensive to enter, narrower in what they can cover, or dependent on intrusive monitoring. Verification could be improvised at short notice, but only crudely. Preparation allows researchers and engineers to develop better options that face fewer trade-offs, and most of this is not slow work if it is resourced seriously: a determined program could accomplish much of this agenda within a year or two.

Research Priorities for AI Compute Verification

James Petrie

August 31, 2026